Token Extensions / Token-2022 讲义

Solana Token Program 进阶功能、完整代码与实作理解笔记

本讲义按照统一 LaTeX 风格整理 **Token Extensions / Token-2022** 的核心知识与代码实作，覆盖：

基础概念、程序差异、Mint 扩展结构、Transfer Fee、Metadata Pointer、 链上元数据、Non-Transferable、完整 TypeScript 示例、常见错误与设计判断。

Detailed Notes / Full Code Reference

2026 年 3 月 22 日

目录

1 章节导读 **3**

2 什么是 Token Extensions / Token-2022 **3**
- 2.1 最核心的一句话 . . . 3
- 2.2 为什么不直接升级原始 Token Program . . . 4
- 2.3 程序地址 . . . 4

3 Token Program 与 Token-2022 的差异 **4**
- 3.1 概念层面的差异 . . . 4
- 3.2 工程层面的差异 . . . 4

4 常见 Extension 总览 **5**

5 扩展到底加在哪里：Mint 还是 Token Account **5**
- 5.1 这是最容易搞混的地方 . . . 5
- 5.2 Mint 级别扩展 . . . 5
- 5.3 Token Account 级别扩展 . . . 6

6 为什么初始化顺序不能乱 **6**
- 6.1 顺序背后的本质 . . . 6
- 6.2 为什么先扩展后 Mint . . . 6

7 项目结构建议 **6**
- 7.1 建议目录 . . . 6
- 7.2 推荐安装套件 . . . 7

8 完整代码一：基础配置文件 **7**
- 8.1 这一段代码为什么重要 . . . 7

9 完整代码二：创建带 Transfer Fee 的 Token-2022 Mint **8**
- 9.1 这段代码在做什么 . . . 9
- 9.2 为什么 `getMintLen()` 很关键 . . . 9
- 9.3 `feeBasisPoints` 与 `maxFee` 的含义 . . . 10

10 完整代码三：给 Token-2022 Mint 铸币 **10**
- 10.1 为什么这里还要显式传入 Program ID . . . 11
- 10.2 为什么 `amount` 要换算成 `rawAmount` . . . 11

11 完整代码四：使用 Token-2022 进行转账并查看手续费 **12**
- 11.1 这段代码为什么先读 Mint 信息 . . . 13
- 11.2 `getTransferFeeAmount()` 的意义 . . . 13
- 11.3 为什么接收方 ATA 也要创建 . . . 14

12 完整代码五：创建带 Metadata Pointer 与链上元数据的 Mint **14**

12.1 Metadata Pointer 到底是什么 16
12.2 为什么这里把 metadata 指向 Mint 自己 16
12.3 为什么空间计算更复杂 17
12.4 为什么这类设计有价值 17

13 完整代码六：创建 Non-Transferable Token（SBT 风格） 17
13.1 Non-Transferable 的设计意义 18
13.2 为什么 SBT 场景常用它 19

14 命令执行方式示例 19
14.1 实作建议顺序 19

15 代码背后的完整逻辑地图 19
15.1 从创建到使用的完整生命周期 20
15.2 这个流程图告诉你什么 20

16 常见错误与踩坑点 20
16.1 忘记传 TOKEN_2022_PROGRAM_ID 20
16.2 Mint 空间不够 20
16.3 初始化顺序错误 21
16.4 金额精度处理粗糙 21

17 如何判断该不该用 Token-2022 21
17.1 适合用 Token-2022 的情况 21
17.2 不一定适合的情况 21

18 学习这部分时最该建立的三个观念 22
18.1 观念一：扩展不是装饰，而是账户结构 22
18.2 观念二：程序 ID 不是小细节，而是运行语义 22
18.3 观念三：功能越强，设计判断越重要 22

19 总结 23

1 章节导读

这份讲义不是只告诉你「Token-2022 可以做更多功能」，而是要把你真正写代码时会碰到的几个核心问题讲清楚：**为什么需要它、它和原始 SPL Token 到底差在哪里、扩展是加在 Mint 还是 Token Account、初始化顺序为什么不能乱、以及每一个扩展背后的设计意义是什么。**

本讲义会按下面的顺序展开：

1. 先建立 Token Program 与 Token-2022 的整体地图。
2. 再解释 Token Extension 的账户结构与初始化逻辑。
3. 接着给出可直接运行的 TypeScript 完整代码。
4. 然后逐段解释代码为什么要这样写。
5. 最后总结适用场景、兼容性问题与常见报错。

你可以把这份讲义理解成：**一份偏工程实战的 Token-2022 学习包**。它不是只背定义，而是尽量让你知道「下一次自己从零开一个项目时，应该怎么搭、怎么想、怎么避免踩坑」。

2 什么是 Token Extensions / Token-2022

2.1 最核心的一句话

Token-2022 可以理解为：**在保留原始 SPL Token 核心能力的基础上，额外支持一系列可选扩展（extensions）的新 Token Program**。

原始 Token Program 擅长的是最基本的代币生命周期：

- 创建 Mint
- 创建 Token Account
- Mint To
- Transfer
- Burn
- Approve / Revoke

而 Token-2022 的重点是：

- 让代币本身在协议层就能带有更复杂的行为
- 把一些过去必须靠外部程序才能做的限制或逻辑，尽量下沉到 Token Program 层
- 用「扩展」方式按需启用功能，而不是强迫所有代币都承担额外复杂度

2.2 为什么不直接升级原始 Token Program

因为原始 SPL Token 已经被大量钱包、协议、交易所与前端工具广泛依赖。若直接在老程序上做破坏式升级，会影响整个生态兼容性。

因此 Solana 的做法是：

- 保留原始 Token Program 继续服务基础代币场景
- 另开一个独立的 Token-2022 Program
- 让开发者按需求选择「极高兼容性」或「更强可编程性」

因此，**Token-2022 不是取代原本 Token Program，而是并行存在的增强版本**。这也是为什么你在代码中常常必须显式传入 `TOKEN_2022_PROGRAM_ID`。

2.3 程序地址

Token-2022 Program 地址：
`TokenzQdBNbLqP5VEhdkAS6EPFLC1PHnBqCXEpPxuEb`

3 Token Program 与 Token-2022 的差异

3.1 概念层面的差异

比较项	原始 Token Program	Token-2022
核心定位	基础代币标准	可扩展代币标准
功能范围	转账、铸造、销毁、授权等基础功能	基础功能 + 多种扩展功能
兼容性	最高，生态最成熟	支持持续增加，但部分协议仍有限制
账户大小	标准大小即可	需预留扩展空间，账户通常更大
初始化复杂度	相对简单	更复杂，顺序要严格正确
适合场景	通用代币、教学入门、最广支持	合规代币、手续费代币、SBT、链上元数据、隐私功能

3.2 工程层面的差异

你在代码里真正会感受到的差异，主要有四个：

1. **程序 ID 不同**：很多 API 必须手动传入 Token-2022 Program ID。
2. **账户空间更大**：因为扩展需要额外 bytes，Mint account 不再是固定最小大小。
3. **初始化顺序更严格**：先建账户，再初始化扩展，再初始化 Mint，本质上是先铺结构再写主数据。

4. **兼容性判断更重要**：不是每个钱包、DEX、协议都完整支持全部扩展。

很多初学者最常见的误区是：**以为把原本 Token Program 的代码复制过来，只要把 program id 改成 Token-2022 就够了**。实际上并不是。扩展意味着账户大小、指令顺序、authority 设计、前端兼容性都可能发生变化。

4 常见 Extension 总览

扩展名称	功能说明	典型应用场景
Transfer Fee	每次转账时自动计算并扣除手续费	协议收入、通缩设计、手续费型代币
Interest Bearing	余额可按规则体现计息效果	储蓄型产品、收益型记账单位
Non-Transferable	禁止普通转让	身份证明、会员资格、成就徽章、SBT
Permanent Delegate	设定永久委托角色	合规资产、可监管资产、托管型代币
Metadata Pointer	指向元数据账户	链上/链外元数据定位与扩展管理
Token Metadata	直接以标准接口保存代币元数据	轻量级链上元数据方案
Transfer Hook	转账时触发自定义逻辑	版税、KYC 检查、业务逻辑拦截
Confidential Transfer	隐藏金额相关信息	隐私场景、企业级资产流转

并不是所有扩展都可以任意组合。某些扩展在语义上互相冲突，所以设计代币之前必须先决定业务目标，而不是看到什么功能就全部往上加。

5 扩展到底加在哪里：Mint 还是 Token Account

5.1 这是最容易搞混的地方

很多人第一次学 Token Extensions，会误以为扩展只是一个「标签」，像前端配置一样挂在 token 上。其实不是。

扩展本质上是：**账户数据布局的扩展**。

也就是说，扩展会反映到链上账户的实际 data layout 里。因此你必须先决定：

- 这个扩展是属于 **Mint 级别**，还是
- 属于 **Token Account 级别**

5.2 Mint 级别扩展

Mint 级别扩展用于定义「这个代币整体」的属性，例如：

- Transfer Fee Config

- Metadata Pointer
- Token Metadata
- Non-Transferable
- Permanent Delegate

这些扩展影响的是整个 token mint 的行为规则，而不是某一个持币人的单独账户。

5.3 Token Account 级别扩展

Token Account 级别扩展用于影响某个持仓账户本身的状态或权限，属于「账户维度」的增强。

判断方法很简单：**如果它定义的是整个代币规则，通常在 Mint；如果它定义的是某个持有账户的局部行为，通常在 Token Account**。

6 为什么初始化顺序不能乱

6.1 顺序背后的本质

因为 Token-2022 的扩展需要预留额外数据空间，所以你在创建 Mint account 时，必须一开始就把空间算对。

典型顺序如下：

1. 先决定要启用哪些 extensions
2. 根据 extensions 计算 Mint account 所需空间
3. 创建账户并注入足够租金豁免 lamports
4. 先初始化相关 extension
5. 最后初始化 Mint 本体

6.2 为什么先扩展后 Mint

因为 Mint 的主结构只是账户的一部分，而扩展是额外布局。若主 Mint 先写入，但扩展区域没有正确准备，后续初始化会失败或出现不一致。

一个非常重要的工程规则：**先分配完整空间，再按顺序写扩展，再初始化 Mint**。如果顺序颠倒，最常见后果就是指令失败、空间不足、或账户状态不合法。

7 项目结构建议

7.1 建议目录

```
solana-token-extensions/
 package.json
 tsconfig.json
 .env
 src/
     config.ts
     create-token-with-fee.ts
     mint-token-2022.ts
     transfer-token-2022.ts
     create-token-with-metadata.ts
     create-non-transferable-token.ts
```

7.2 推荐安装套件

```
npm install @solana/web3.js @solana/spl-token @solana/spl-token-metadata dotenv
npm install -D typescript ts-node @types/node
```

这里的重点不是把包装上就结束，而是你要意识到：`@solana/spl-token` 在 Token-2022 场景下会提供更多 extension 相关 API；而 `@solana/spl-token-metadata` 则用于处理 Token Metadata Interface 的相关指令与序列化。

8 完整代码一：基础配置文件

下面先给出统一使用的 `config.ts`，后续所有脚本都依赖它。

Listing 1: src/config.ts

```typescript
import 'dotenv/config';
import { Connection, Keypair, clusterApiUrl } from '@solana/web3.js';
import fs from 'fs';
import path from 'path';

const RPC_URL = process.env.RPC_URL || clusterApiUrl('devnet');
const WALLET_PATH =
  process.env.WALLET_PATH ||
  path.join(process.env.HOME || '.', '.config/solana/id.json');

export const connection = new Connection(RPC_URL, 'confirmed');

export function loadWallet(): Keypair {
  const secret = JSON.parse(fs.readFileSync(WALLET_PATH, 'utf-8')) as number[];
  return Keypair.fromSecretKey(Uint8Array.from(secret));
}
```

8.1 这一段代码为什么重要

1. 它把 RPC 与钱包载入逻辑统一起来，避免每个脚本都重复写。

2. 它让你可以通过环境变量自由切换不同 RPC 或钱包。
3. 它使后续脚本更聚焦在「Token-2022 逻辑」，而不是环境样板代码。

当你后面写 `const payer = loadWallet();` 时，本质上就是在把本地钱包文件转成可签名的 `Keypair` 对象。没有这一步，你的交易无法完成签名。

9 完整代码二：创建带 Transfer Fee 的 Token-2022 Mint

Listing 2: src/create-token-with-fee.ts

```
import {
  ExtensionType,
  TOKEN_2022_PROGRAM_ID,
  createInitializeMintInstruction,
  createInitializeTransferFeeConfigInstruction,
  getMintLen,
} from '@solana/spl-token';
import {
  Keypair,
  SystemProgram,
  Transaction,
  sendAndConfirmTransaction,
} from '@solana/web3.js';
import { connection, loadWallet } from './config';

async function createTokenWithTransferFee() {
  const payer = loadWallet();
  const mintKeypair = Keypair.generate();

  const decimals = 6;
  const feeBasisPoints = 100;      // 1%
  const maxFee = BigInt(1_000_000); // 1 token if decimals = 6

  const extensions = [ExtensionType.TransferFeeConfig];
  const mintLen = getMintLen(extensions);
  const lamports = await connection.getMinimumBalanceForRentExemption(mintLen);

  console.log('开始创建 Token-2022 Mint...');
  console.log('Mint:', mintKeypair.publicKey.toBase58());
  console.log('Mint account size:', mintLen);
  console.log('Rent-exempt lamports:', lamports);

  const tx = new Transaction().add(
    SystemProgram.createAccount({
      fromPubkey: payer.publicKey,
      newAccountPubkey: mintKeypair.publicKey,
      space: mintLen,
```

```
      lamports,
      programId: TOKEN_2022_PROGRAM_ID,
    }),
    createInitializeTransferFeeConfigInstruction(
      mintKeypair.publicKey,
      payer.publicKey, // transfer fee config authority
      payer.publicKey, // withdraw withheld authority
      feeBasisPoints,
      maxFee,
      TOKEN_2022_PROGRAM_ID
    ),
    createInitializeMintInstruction(
      mintKeypair.publicKey,
      decimals,
      payer.publicKey, // mint authority
      null,            // freeze authority
      TOKEN_2022_PROGRAM_ID
    )
  );

  const signature = await sendAndConfirmTransaction(connection, tx, [payer,
    mintKeypair]);

  console.log('创建完成');
  console.log('Mint Address:', mintKeypair.publicKey.toBase58());
  console.log('Signature:', signature);
}

createTokenWithTransferFee().catch(console.error);
```

9.1 这段代码在做什么

这段脚本的真实工作流程可以拆成五件事：

1. 生成一个新的 Mint keypair
2. 指定这个 Mint 要启用 `TransferFeeConfig` 扩展
3. 根据扩展重新计算账户大小
4. 创建链上账户并分配 rent-exempt lamports
5. 先初始化手续费扩展，再初始化 Mint

9.2 为什么 `getMintLen()` 很关键

因为在 Token-2022 中，Mint 不再是简单固定大小。如果你启用了扩展，账户空间一定要跟着变大。

如果你忘记把扩展放进 `getMintLen(extensions)`，最常见结果就是：**账户虽然创建成功，但后续 extension initialization 会因为空间不足而失败**。

9.3 feeBasisPoints 与 maxFee 的含义

- `feeBasisPoints`：手续费率，单位是 basis points（万分比）
- `100` 表示 1%
- `maxFee`：每笔交易手续费上限，防止大额转账产生过高手续费

举例来说：

- 若转账 200 个 token，费率 1%，理论手续费是 2 个 token
- 但如果设了上限 1 token，那么最终只收 1 token

这说明 Transfer Fee 扩展不是单纯「乘一个百分比」而已，而是「**费率 + 上限**」共同决定真实收费结果。这种设计更适合实际产品场景。

10 完整代码三：给 Token-2022 Mint 铸币

Listing 3: src/mint-token-2022.ts

```
import {
  TOKEN_2022_PROGRAM_ID,
  getOrCreateAssociatedTokenAccount,
  mintTo,
} from '@solana/spl-token';
import { PublicKey } from '@solana/web3.js';
import { connection, loadWallet } from './config';

async function mintToken2022(mintAddress: string, amount: number) {
  const payer = loadWallet();
  const mint = new PublicKey(mintAddress);

  const decimals = 6;
  const rawAmount = BigInt(amount * 10 ** decimals);

  const tokenAccount = await getOrCreateAssociatedTokenAccount(
    connection,
    payer,
    mint,
    payer.publicKey,
    false,
    'confirmed',
    undefined,
    TOKEN_2022_PROGRAM_ID
  );
```

```
  const signature = await mintTo(
    connection,
    payer,
    mint,
    tokenAccount.address,
    payer,
    rawAmount,
    [],
    undefined,
    TOKEN_2022_PROGRAM_ID
  );

  console.log('Mint 成功');
  console.log('ATA:', tokenAccount.address.toBase58());
  console.log('Signature:', signature);
}

mintToken2022('REPLACE_WITH_MINT_ADDRESS', 1000).catch(console.error);
```

10.1 为什么这里还要显式传入 Program ID

因为 Associated Token Account 的导出地址、账户校验、以及实际 mint / transfer 逻辑，都必须知道你是在和哪一个 token program 交互。

若你创建的是 Token-2022 Mint，但后续调用仍默认走旧 Token Program，就会出现：

- 找不到正确账户
- 账户 owner / program 校验失败
- mint / transfer 指令报错

10.2 为什么 `amount` 要换算成 `rawAmount`

链上不会直接存「1.5 个 token」这样的浮点数，而是存最小单位整数。

若 `decimals = 6`，那么：

$$1 \text{ token} = 10^6 \text{ raw units}$$

所以当你写：

```
const rawAmount = BigInt(amount * 10 ** decimals);
```

本质上是把人类可读金额转成链上整数金额。

> 生产环境里，若金额可能很大或来自小数输入，建议避免直接用 JavaScript `number` 参与高精度计算，而是改用更稳定的字符串或 bigint 处理方式，以免出现精度问题。

11 完整代码四：使用 Token-2022 进行转账并查看手续费

Listing 4: src/transfer-token-2022.ts

```
import {
  TOKEN_2022_PROGRAM_ID,
  getMint,
  getOrCreateAssociatedTokenAccount,
  getTransferFeeAmount,
  transfer,
} from '@solana/spl-token';
import { PublicKey } from '@solana/web3.js';
import { connection, loadWallet } from './config';

async function transferWithFee(
  mintAddress: string,
  recipientWallet: string,
  amount: number
) {
  const payer = loadWallet();
  const mint = new PublicKey(mintAddress);
  const recipient = new PublicKey(recipientWallet);

  const mintInfo = await getMint(
    connection,
    mint,
    'confirmed',
    TOKEN_2022_PROGRAM_ID
  );

  const rawAmount = BigInt(amount * 10 ** mintInfo.decimals);

  const sourceAccount = await getOrCreateAssociatedTokenAccount(
    connection,
    payer,
    mint,
    payer.publicKey,
    false,
    'confirmed',
    undefined,
    TOKEN_2022_PROGRAM_ID
  );

  const destinationAccount = await getOrCreateAssociatedTokenAccount(
    connection,
    payer,
    mint,
    recipient,
```

```
    false,
    'confirmed',
    undefined,
    TOKEN_2022_PROGRAM_ID
  );

  const fee = getTransferFeeAmount(mintInfo, rawAmount, 'newerTransferFee');

  console.log('转账金额:', amount);
  console.log('预计手续费(raw):', fee?.fee.toString() || '0');
  console.log('接收方实收(raw):', (rawAmount - (fee?.fee || 0n)).toString());

  const signature = await transfer(
    connection,
    payer,
    sourceAccount.address,
    destinationAccount.address,
    payer,
    rawAmount,
    [],
    undefined,
    TOKEN_2022_PROGRAM_ID
  );

  console.log('转账成功');
  console.log('Signature:', signature);
}

transferWithFee(
  'REPLACE_WITH_MINT_ADDRESS',
  'REPLACE_WITH_RECIPIENT_WALLET',
  25
).catch(console.error);
```

11.1 这段代码为什么先读 Mint 信息

因为手续费规则是写在 Mint 扩展里的，不是写在用户钱包里。要计算一笔转账应该扣多少费，必须先知道这个 Mint 的 fee config。

11.2 getTransferFeeAmount() 的意义

它的作用不是发交易，而是帮助你在交易前先估算：

- 理论手续费是多少
- 接收方大概会收到多少净额

这对于前端 UX 很重要，因为用户通常想在点击发送前就看到：

- 总转账金额
- 手续费
- 对方实际收到金额

也就是说，Transfer Fee 扩展不只是链上逻辑问题，还会直接影响你的前端显示、交易确认页、收据页与会计逻辑。

11.3 为什么接收方 ATA 也要创建

因为 token 不是直接发到「主钱包地址」本体，而是发到该钱包针对某个 Mint 对应的 Token Account / ATA。

如果接收方没有这个 Mint 的 ATA，就必须先创建，否则 token 无处可放。

12 完整代码五：创建带 Metadata Pointer 与链上元数据的 Mint

Listing 5: src/create-token-with-metadata.ts

```
import {
  ExtensionType,
  TOKEN_2022_PROGRAM_ID,
  TYPE_SIZE,
  LENGTH_SIZE,
  createInitializeMintInstruction,
  createInitializeMetadataPointerInstruction,
  getMintLen,
} from '@solana/spl-token';
import {
  TokenMetadata,
  pack,
  createInitializeInstruction,
  createUpdateFieldInstruction,
} from '@solana/spl-token-metadata';
import {
  Keypair,
  SystemProgram,
  Transaction,
  sendAndConfirmTransaction,
} from '@solana/web3.js';
import { connection, loadWallet } from './config';

async function createTokenWithMetadata() {
  const payer = loadWallet();
  const mintKeypair = Keypair.generate();

  const metadata: TokenMetadata = {
    mint: mintKeypair.publicKey,
```

```
    name: 'Future Credit',
    symbol: 'FCR',
    uri: 'https://example.com/fcr.json',
    additionalMetadata: [
      ['description', 'Utility token using Token-2022 metadata extensions'],
      ['project', 'Future DAO'],
      ['website', 'https://example.com']
    ],
  };

  const metadataExtensionSize = TYPE_SIZE + LENGTH_SIZE;
  const metadataLen = pack(metadata).length;

  const extensions = [ExtensionType.MetadataPointer];
  const mintLen = getMintLen(extensions) + metadataExtensionSize + metadataLen;
  const lamports = await connection.getMinimumBalanceForRentExemption(mintLen);

  const tx = new Transaction().add(
    SystemProgram.createAccount({
      fromPubkey: payer.publicKey,
      newAccountPubkey: mintKeypair.publicKey,
      space: mintLen,
      lamports,
      programId: TOKEN_2022_PROGRAM_ID,
    }),
    createInitializeMetadataPointerInstruction(
      mintKeypair.publicKey,
      payer.publicKey,
      mintKeypair.publicKey,
      TOKEN_2022_PROGRAM_ID
    ),
    createInitializeMintInstruction(
      mintKeypair.publicKey,
      6,
      payer.publicKey,
      null,
      TOKEN_2022_PROGRAM_ID
    ),
    createInitializeInstruction({
      programId: TOKEN_2022_PROGRAM_ID,
      metadata: mintKeypair.publicKey,
      mint: mintKeypair.publicKey,
      mintAuthority: payer.publicKey,
      updateAuthority: payer.publicKey,
      name: metadata.name,
      symbol: metadata.symbol,
      uri: metadata.uri,
    })
```

```
  );

  for (const [key, value] of metadata.additionalMetadata) {
    tx.add(
      createUpdateFieldInstruction({
        programId: TOKEN_2022_PROGRAM_ID,
        metadata: mintKeypair.publicKey,
        updateAuthority: payer.publicKey,
        field: key,
        value,
      })
    );
  }

  const signature = await sendAndConfirmTransaction(connection, tx, [payer,
    mintKeypair]);

  console.log('创建成功');
  console.log('Mint:', mintKeypair.publicKey.toBase58());
  console.log('Signature:', signature);
}

createTokenWithMetadata().catch(console.error);
```

12.1 Metadata Pointer 到底是什么

它不是元数据本身，而是：**告诉别人这个 Mint 的元数据应该去哪里找**。

你可以把它理解成：

- Metadata Pointer = 指针 / 导航地址
- Token Metadata = 具体元数据内容

12.2 为什么这里把 metadata 指向 Mint 自己

因为这份示例采用的是一种轻量方案：**把元数据直接存到 Mint 账户自己身上**。

所以这里会出现：

```
metadata: mintKeypair.publicKey,
mint: mintKeypair.publicKey,
```

也就是：

- 这个账户既是 Mint
- 也是 metadata 存放位置

12.3 为什么空间计算更复杂

因为这里不只要装下 Mint + MetadataPointer 扩展，还要额外装：

- metadata type / length 编码信息
- name / symbol / uri
- additional metadata fields

所以这里空间计算是：

```
const mintLen = getMintLen(extensions) + metadataExtensionSize + metadataLen;
```

如果你只调用 `getMintLen([ExtensionType.MetadataPointer])`，但没有把实际 metadata 内容长度加进去，创建账户后通常还是不够大，后续初始化 metadata 时就会失败。

12.4 为什么这类设计有价值

在过去，很多 token 的 metadata 需要依赖额外程序管理。Token-2022 的 metadata 相关扩展让某些场景可以用更轻量、更直接的方式完成元数据表达。

但你也要理解：**轻量并不总是等于最适合**。如果你的场景需要复杂 NFT 生态兼容、成熟索引、丰富标准支持，仍然要评估别的元数据方案。

13 完整代码六：创建 Non-Transferable Token（SBT 风格）

Listing 6: src/create-non-transferable-token.ts

```
import {
  ExtensionType,
  TOKEN_2022_PROGRAM_ID,
  createInitializeMintInstruction,
  createInitializeNonTransferableMintInstruction,
  getMintLen,
} from '@solana/spl-token';
import {
  Keypair,
  SystemProgram,
  Transaction,
  sendAndConfirmTransaction,
} from '@solana/web3.js';
import { connection, loadWallet } from './config';

async function createNonTransferableMint() {
  const payer = loadWallet();
  const mintKeypair = Keypair.generate();

  const extensions = [ExtensionType.NonTransferable];
```

```
  const mintLen = getMintLen(extensions);
  const lamports = await connection.getMinimumBalanceForRentExemption(mintLen);

  const tx = new Transaction().add(
    SystemProgram.createAccount({
      fromPubkey: payer.publicKey,
      newAccountPubkey: mintKeypair.publicKey,
      space: mintLen,
      lamports,
      programId: TOKEN_2022_PROGRAM_ID,
    }),
    createInitializeNonTransferableMintInstruction(
      mintKeypair.publicKey,
      TOKEN_2022_PROGRAM_ID
    ),
    createInitializeMintInstruction(
      mintKeypair.publicKey,
      0,
      payer.publicKey,
      null,
      TOKEN_2022_PROGRAM_ID
    )
  );

  const signature = await sendAndConfirmTransaction(connection, tx, [payer,
    mintKeypair]);

  console.log('Non-transferable mint created');
  console.log('Mint:', mintKeypair.publicKey.toBase58());
  console.log('Signature:', signature);
}

createNonTransferableMint().catch(console.error);
```

13.1 Non-Transferable 的设计意义

这是 Token-2022 很有代表性的扩展之一，因为它改变的不是余额数值，而是代币的「可转移性」。

当一个 Mint 被定义成 Non-Transferable 时，普通用户间的 token transfer 不再成立。这很适合表示：

- 身份凭证
- 课程结业证书
- DAO 会员资格

- 贡献记录徽章

这类 token 的重点不是金融流通，而是**身份与状态证明**。因此它更接近「证明型资产」而不是「交易型资产」。

13.2 为什么 SBT 场景常用它

因为 SBT（Soulbound Token）最重要的特征就是：

- 发给谁，就是谁的证明
- 不应该被随意卖掉或转走

若一个会员资格 token 可以自由交易，那么会员身份就会变得不可靠。Non-Transferable 扩展正好能从协议层限制这种行为。

14 命令执行方式示例

```
npx ts-node src/create-token-with-fee.ts
npx ts-node src/mint-token-2022.ts
npx ts-node src/transfer-token-2022.ts
npx ts-node src/create-token-with-metadata.ts
npx ts-node src/create-non-transferable-token.ts
```

14.1 实作建议顺序

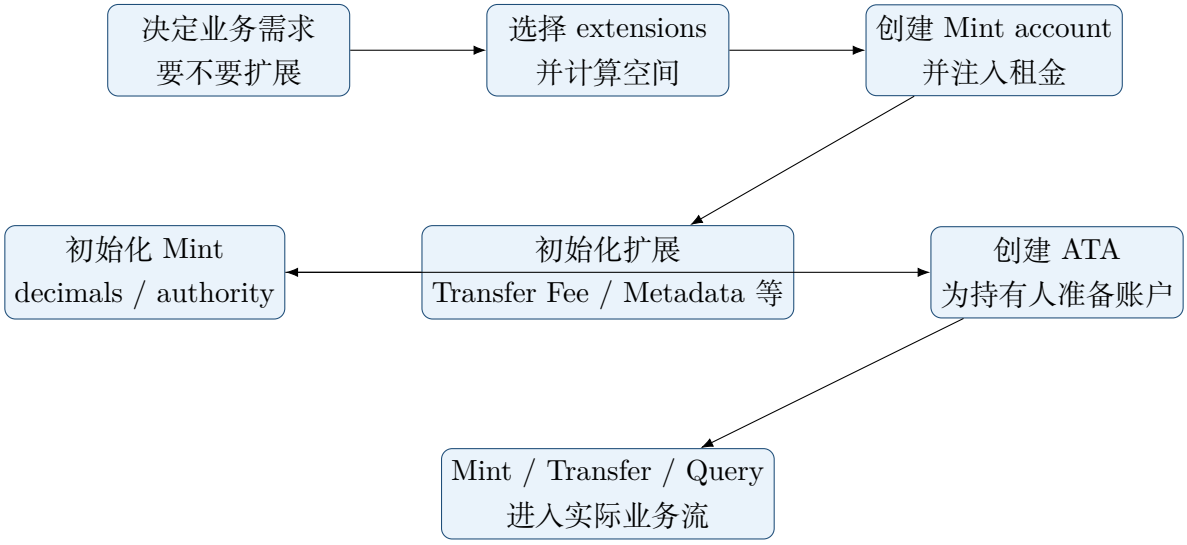
如果你第一次从零操作，建议顺序如下：

1. 先创建带 Transfer Fee 的 Mint
2. 记录 Mint 地址
3. 给自己 mint 一批 token
4. 找第二钱包地址做 transfer 测试
5. 再测试 metadata 与 non-transferable 场景

不要一开始就把所有扩展混在一起测试。最稳的方法永远是：**先单扩展验证，再逐步叠加复杂度**。

15 代码背后的完整逻辑地图

15.1 从创建到使用的完整生命周期



15.2 这个流程图告诉你什么

它告诉你：**Token-2022 不是单一指令，而是一套账户布局 + 初始化策略 + 使用阶段规则的整体设计**。

当你理解这点后，再去看每一个 API，就不会只把它们当成零碎命令，而会知道它们属于哪一层：

- 有些负责分配空间
- 有些负责写规则
- 有些负责实际流转

16 常见错误与踩坑点

16.1 忘记传 `TOKEN_2022_PROGRAM_ID`

这是最经典错误之一。

症状通常包括：

- 找不到正确的 ATA
- transfer / mint 失败
- 账户 owner 校验报错

根因：你创建的是 Token-2022 的 Mint，但后续函数默认仍按老 Token Program 处理。

16.2 Mint 空间不够

症状通常包括：

- 初始化扩展时报错
- metadata 初始化失败
- 程序提示 invalid account data / insufficient space

根因：只算了基础 Mint 大小，没有把扩展和 metadata 序列化长度算进去。

16.3 初始化顺序错误

症状通常包括：

- 某个 extension instruction 无法执行
- Mint 已初始化，但扩展状态不完整

根因：先初始化 Mint，再去初始化扩展，破坏了预期顺序。

16.4 金额精度处理粗糙

症状通常包括：

- 铸币金额不对
- 前端展示与链上余额不一致
- 大金额时出现精度偏差

根因：直接用 JavaScript 浮点数做金额换算。

你越往正式产品做，越不能把金额运算当成小事。**Token 金额、手续费、净额、上限、UI 显示与链上原始值一定要统一口径。**

17 如何判断该不该用 Token-2022

17.1 适合用 Token-2022 的情况

- 你明确需要 Transfer Fee、Non-Transferable、Metadata Pointer 等扩展
- 你的业务规则希望尽量在 token program 层表达
- 你愿意为了功能性接受稍高的复杂度与兼容性评估成本

17.2 不一定适合的情况

- 你只需要最普通的可转账代币
- 你极度依赖现有钱包 / DEX / 协议的最广泛兼容
- 你目前主要目标是先完成最简 MVP

判断问题	若答案偏向「是」	建议
是否需要协议层手续费	是	考虑 Transfer Fee
是否需要身份类代币	是	考虑 Non-Transferable
是否需要轻量链上元数据	是	考虑 Metadata Pointer + Token Metadata
是否最在意生态兼容性	是	优先考虑原始 Token Program
是否只是一般 fungible token	是	先用旧 Token Program 也合理

真正的设计思路不是「Token-2022 比较新所以一定比较好」，而是：**你的业务是否真的需要这些扩展带来的协议层能力**。

18 学习这部分时最该建立的三个观念

18.1 观念一：扩展不是装饰，而是账户结构

只要你脑中还把 extension 想成「附加标签」，就会一直不理解为什么要重新算空间、为什么初始化顺序这么严格。

18.2 观念二：程序 ID 不是小细节，而是运行语义

不同 program id 不只是地址不同，而是代表：

- 你在和哪个程序交互
- 该程序支持哪些指令
- 账户所有权与解析规则是什么

18.3 观念三：功能越强，设计判断越重要

Token-2022 让你能做更多，但也意味着你要更认真思考：

- authority 应该给谁
- 能否升级或修改
- 前端与钱包如何展示
- 生态工具是否支持
- 哪些扩展可共存，哪些会冲突

19 总结

Token Extensions / Token-2022 的本质，不是「多了几个 API」，而是：**Solana 允许你在 token 标准层做更细致的行为建模**。

如果你只记住这份讲义中的几件事，请至少记住下面这些：

1. Token-2022 与原始 Token Program 并行存在。
2. 扩展会影响账户数据布局，所以必须先算空间。
3. 初始化顺序通常是：创建账户 → 初始化扩展 → 初始化 Mint。
4. 很多函数必须显式传入 `TOKEN_2022_PROGRAM_ID`。
5. Transfer Fee、Metadata、Non-Transferable 各自对应完全不同的业务语义。
6. 是否使用 Token-2022，不是看新不新，而是看业务需不需要。

当你能把这几件事讲清楚时，你对 Token-2022 的理解就已经不是「会照着教程敲」，而是开始进入「可以自己设计与判断」的阶段了。